Lab 3 : Game Ai Using Minimax with Alpha-Beta Pruning

Fundamentals of Artificial Intellingence

Name: Rey Vincent B. Ilosorio

Course: BSCS AI

Section: 09282-FUNDAI

Date: September 1, 2026

Selected Game: Tic-Tac-Toe

GitHub URL: https://github.com/ReyVincent-eng/ FUNDAI-Laboratories-ILOSORIO

Description

This laboratory implements a Tic-Tac-Toe Ai using Minimax with Alpha-Beta pruning. The game is playable inside Goodle Colab using ipywidgets

```
import math
import ipywidgets as widgets
from IPython.display import display

class TicTacToeGame:
    X = "X"
    O = "O"
    EMPTY = " "

    def __init__(self):
        self.board = [self.EMPTY] * 9
        self.current_player = self.X

    def available_moves(self):
        return [i for i, value in enumerate(self.board) if value == self.EMPTY]

    def make_move(self, move):
        self.board[move] = self.current_player
        self.current_player = self.O if self.current_player == self.X else self.X

    def undo_move(self, move):
        self.board[move] = self.EMPTY
        self.current_player = self.O if self.current_player == self.X else self.X

    def get_winner(self):
        winning_lines = [
            (0, 1, 2), (3, 4, 5), (6, 7, 8),
            (0, 3, 6), (1, 4, 7), (2, 5, 8),
            (0, 4, 8), (2, 4, 6)
        ]
        for a, b, c in winning_lines:
            if self.board[a] != self.EMPTY and self.board[a] == self.board[b] == self.bo
                return self.board[a]
        return None
```

```
    def is_draw(self):
        return self.get_winner() is None and len(self.available_moves()) == 0

    def is_terminal(self):
        return self.get_winner() is not None or len(self.available_moves()) == 0

    def utility(self):
        winner = self.get_winner()
        if winner == self.X:
            return 1
        elif winner == self.O:
            return -1
        else:
            return 0

    def minimax_alpha_beta(self, alpha=-math.inf, beta=math.inf):
        if self.is_terminal():
            return self.utility(), None

        if self.current_player == TicTacToeGame.X:
            best_value = -math.inf
            best_move = None
            for move in self.available_moves():
                self.make_move(move)
                value, _ = self.minimax_alpha_beta(alpha, beta)
                self.undo_move(move)
                if value > best_value:
                    best_value = value
                    best_move = move
                alpha = max(alpha, best_value)
                if alpha >= beta:
                    break
            return best_value, best_move
        else:
            best_value = math.inf
            best_move = None
            for move in self.available_moves():
                self.make_move(move)
                value, _ = self.minimax_alpha_beta(alpha, beta)
                self.undo_move(move)
                if value < best_value:
                    best_value = value
                    best_move = move
                beta = min(beta, best_value)
                if alpha >= beta:
                    break
            return best_value, best_move


class TicTacToeUI:
    def __init__(self):
        self.game = TicTacToeGame()

        self.buttons = [
            widgets.Button(description=" ", layout=widgets.Layout(width="60px", height="
            for i in range(9)
        ]

        for i in range(9):
            self.buttons[i].on_click(lambda btn, idx=i: self.on_cell_click(idx))
```

```
        self.status = widgets.HTML(value="<b>Human X moves first.</b>")

        self.reset_buttons = widgets.Button(description="Reset", button_style="info")
        self.reset_buttons.on_click(self.on_reset)

        self.grid = widgets.GridBox(
            children=self.buttons,
            layout=widgets.Layout(
                grid_template_columns="repeat(3, 60px)",
                grid_gap="5px"
            )
        )

        self.widget = widgets.VBox([self.status, self.grid, self.reset_buttons])
        display(self.widget)

        self.refresh()

    def refresh(self):
        for i, button in enumerate(self.buttons):
            button.description = self.game.board[i]
            button.disabled = self.game.is_terminal() or self.game.board[i] != TicTacToe

        if self.game.is_terminal():
            winner = self.game.get_winner()
            if winner:
                self.status.value = f"<b>Player {winner} wins!</b>"
            else:
                self.status.value = "<b>Draw!</b>"
        else:
            self.status.value = f"<b>Current player: {self.game.current_player}</b>"

    def on_cell_click(self, index):
        if self.game.board[index] != TicTacToeGame.EMPTY:
            return
        if self.game.is_terminal():
            return
        if self.game.current_player != TicTacToeGame.X:
            return

        self.game.make_move(index)

        if not self.game.is_terminal():
            ai_move = self.get_ai_move()
            if ai_move is not None:
                self.game.make_move(ai_move)

        self.refresh()

    def get_ai_move(self):
        _, move = self.game.minimax_alpha_beta()
        return move

    def on_reset(self, button):
        self.game = TicTacToeGame()
        self.refresh()


TicTacToeUI()
```

```
Draw!
    X       O       X

    O       O       X

    X       X       O

          Reset
<__main__.TicTacToeUI at 0x7aeaa9c63b60>
```

Algorithm Explanation

Minimax

Player X maximizes utility, while Player O minimizes utility.

Alpha-Beta Pruning

Alpha-Beta pruning removes branches that cannot change the final decision. This makes the AI faster while preserving the optimal result.

Utility

- X wins: +1 Draw: 0
- O wins: -1

Guide Questions and Answers

1. Which player does the AI control?

Answer: The AI controls Player O. It moves automatically after the human Player X makes a move.

2. What utility values were used?

Answer: X wins = +1, Draw = 0, O wins = -1. These values help the AI choose the best move.

3. How does Alpha-Beta pruning improve Minimax?

Answer: Alpha-Beta pruning skips branches that cannot affect the final decision. This makes Minimax faster while still finding the optimal move.

4. What happens when the human chooses a move that leads to a draw?

Answer: The game ends in a draw when the board is full and neither player wins. The status displays “Draw!”.

5. Why is Tic-Tac-Toe suitable for full Minimax search?

Answer: Tic-Tac-Toe has a small number of possible game states. This allows Minimax to search the game tree completely and find the best move.

Reflection

Challenges Encountered

- One challenge I encountered was understanding how Minimax and Alpha-Beta Pruning work together. I also had to understand how the AI chooses the best move based on the possible outcomes.

What I Learned

- I learned how Minimax helps an AI make the best decision by considering possible moves. I also learned that Alpha-Beta Pruning makes the search faster by removing unnecessary branches.